VIT BHOPAL

PROJECT REPORT OF FUNDAMENTAL IN AI/ML

SUBJECT_CODE: CSA2001

Name: Aviral Verma

Reg.No:25BCE10954

Topic: Job Market Analyser Using Python And Sql

PROJECT REPORT

Job Portal Analyzer using Python and SQLite

---

1. Introduction

In today's digital world, job portals play an important role in connecting job seekers with recruiters. This project, *Job Portal Analyzer*, is a menu-driven application developed using Python and SQLite that simulates a basic job portal system. It allows applicants to apply for jobs and recruiters to search for candidates based on skills and experience.

The system also analyzes current job trends by tracking how frequently specific skills are searched by recruiters.

---

2. Objectives

- To develop a job portal system using Python and SQLite
- To allow applicants to register with multiple skills
- To enable recruiters to filter candidates efficiently
- To analyze and display trending job skills
- To implement database operations like insert, update, delete, and search

---

3. Scope of the Project

This project is suitable for small-scale job portals and academic purposes. It demonstrates how database systems can be used to manage and analyze job-related data.

---

4. Technologies Used

Technology	Description
Python	Used for application logic and menu-driven interface
SQLite	Used for database storage
PrettyTable	Used for displaying output in tabular format

---

5. System Design

5.1 Database Structure

Applicant Table

Field Name	***Type***	***Description***
id	INTEGER	Primary Key (Auto Increment)
name	TEXT	Applicant name
experience	INTEGER	Years of experience
skill	TEXT	Skills (comma-separated)
contact_number	TEXT	Contact number

Skills Table

FIELD NAME	***TYPE***	***DESCRIPTION***
Skill name	TEXT	Skill name (Primary Key)
Search_count	INTEGER	Number of times searched

5.2 System Flow

1. User selects an option from the menu
2. Based on input:
    - Applicant data is stored
    - Recruiter searches candidates
    - Trends are updated and displayed
3. Database operations are performed accordingly

---

6. Functional Modules

6.1 Apply for Job

- Applicant enters name, experience, contact
- Selects multiple skills from predefined list
- Data is stored in applicant table

6.2 Recruit Applicant

- Recruiter selects required skill and experience
- System searches database using SQL query

- Displays list of eligible candidates
- Updates skill search count

6.3 Trend Analyzer

- Displays top 5 most searched skills
- Helps understand market demand

6.4 Delete Application

- Applicant can remove their data
- Requires name and contact number

---

7. Key Code Logic

Insert Data

```
cursor.execute(
    "INSERT INTO applicant (name, experience, skill, contact_number) VALUES (?, ?, ?, ?)",
    (name, exp, skills, contact)
)
```

Search Applicants

```
cursor.execute(
    "SELECT * FROM applicant WHERE skill LIKE ? AND experience >= ?",
    ('%' + skill + '%', exp)
)
```

Update Search Count

```
cursor.execute(
    "UPDATE skills SET search_count = search_count + 1 WHERE skill_name = ?",
    (skill,)
)
```

Delete Applicant

```
cursor.execute(
    "DELETE FROM applicant WHERE name = ? AND contact_number = ?",
    (name, contact)
)
```

---

8. Output

The system provides output in a structured tabular format using PrettyTable, making it easy to read and understand.

---

9. Advantages

- Simple and easy to use
- Efficient data handling using SQLite
- Real-time trend analysis
- Supports multiple skills per applicant

---

10. Limitations

- Command-line interface only
- No user authentication
- Skills stored as text (not normalized)
- Not suitable for large-scale systems

---

11. Future Enhancements

- Graphical User Interface (GUI)
- Web-based implementation
- Resume upload feature
- AI-based candidate recommendation
- Login system for users

---

12. Conclusion

The Job Portal Analyzer project successfully demonstrates how Python and SQLite can be used to build a functional job portal system. It provides a practical understanding of database operations, filtering logic, and real-world problem-solving techniques.

---

13. References

- Python Official Documentation
- SQLite Documentation
- PrettyTable Library Documentation